

ResistiMap

ROS 2 Architecture & Deep Learning Component Reference

1. ROS 2 Components Used in the Project

ResistiMap is built on **ROS 2 Jazzy** (Ubuntu 24.04). The project uses the following core ROS 2 concepts and building blocks, each serving a specific purpose in the system.

1.1 ROS 2 Concepts Used

Concept	Used For
Nodes	Each independent program (motor control, probe control, sensing, classification, visualization, scan management) runs as its own ROS 2 node, so a failure in one does not crash the others.
Topics (Publish/Subscribe)	Used for continuous, one-directional data flow: resistivity readings, risk classifications, motor "go to cell"/"arrived" signals, and probe "lower"/"raise" commands.
Custom Messages (.msg)	Resistivity.msg and RiskClassification.msg - project-specific data structures carried over topics.
Actions (.action)	ScanGrid.action - manages the full grid scan. Chosen over a topic or service because a scan is long-running, needs live progress feedback per cell, and must support mid-scan cancellation.
Packages	Code is organized into three ROS 2 packages: resistimap_msgs , resistimap_hardware , resistimap_perception (see 1.2).
colcon	The build tool used to compile all packages and generate the Python interfaces for the custom messages/action.
rclpy	The Python client library used to write every node (all nodes are Python, matching the Python-based sensor/motor/ML code).
MultiThreadedExecutor / ReentrantCallbackGroup	Used inside the Action server so it can simultaneously host the action, publish motor/probe commands, and listen for sensor data without blocking itself.
rosidl (generators/adapters)	The underlying ROS 2 toolchain that converts the .msg/.action definition files into usable Python classes at build time.

1.2 ROS 2 Packages

Package	Build Type	Contains
resistimap_msgs	ament_cmake	Custom interfaces: Resistivity.msg, RiskClassification.msg, ScanGrid.action

resistimap_hardware	ament_python	motor_driver_node, probe_actuator_node, resistivity_sensor_node, grid_coverage_action_server
resistimap_perception	ament_python	risk_classifier_node, heatmap_builder_node, model training script

1.3 Nodes and Their Communication

Node	Role	Publishes / Subscribes
motor_driver_node	Drives rover to a grid cell	Sub: motor/goto_cell Pub: motor/arrived
probe_actuator_node	Lowers/raises the probes	Sub: probe/command Pub: probe/lowered
resistivity_sensor_node	Reads/computes resistivity	Pub: resistivity (Resistivity.msg)
grid_coverage_action_server	Runs the full grid scan	Hosts: /scan_grid (ScanGrid.action) Talks to all 3 nodes above
risk_classifier_node	DL risk prediction	Sub: resistivity Pub: risk_classification
heatmap_builder_node	Live visualization	Sub: risk_classification

2. The ROS Part of the Project

2.1 Why ROS 2 at All

ResistiMap has several physical subsystems (motors, a probe actuator, a resistivity sensor) that must coordinate with software subsystems (a DL classifier, a heatmap display) without any one of them being tightly coupled to the others. ROS 2 provides this coordination layer: each subsystem runs as an independent node, and nodes communicate only through well-defined messages, topics, and actions - never by calling each other's code directly.

2.2 How a Scan Actually Runs

The **grid_coverage_action_server** is the central coordinator. When it receives a ScanGrid goal (rows, columns, and cell spacing), it performs the following loop for every cell in a lawnmower (boustrophedon) pattern:

- 1 Publish a "go to cell (x, y)" command to motor_driver_node and wait for its "arrived" confirmation.
- 2 Publish a "lower" command to probe_actuator_node and wait for confirmation the probes are down.
- 3 Wait for the next resistivity reading published by resistivity_sensor_node.
- 4 Publish a "raise" command to probe_actuator_node.
- 5 Publish live Feedback on the Action (cells completed, current cell, last resistivity value).
- 6 Move to the next cell, or - once all cells are done - return a Result (success/failure and a summary message).

In parallel with this loop, **risk_classifier_node** and **heatmap_builder_node** are independently subscribed to the resistivity and risk-classification topics. They are not called by the Action server - they simply react whenever new data appears, which is what lets the heatmap update live, cell by cell, during the scan rather

than only at the very end.

2.3 Why an Action (not a Topic or a Service)

A **topic** has no concept of a single "call" with a result - it only streams data. A **service** is a single request/response with no way to report progress while it runs, and no way to cancel it. A full grid scan can take a long time, benefits from showing live per-cell progress during a demo, and should be stoppable if something looks physically wrong with the rover mid-scan. The ROS 2 **Action** pattern was built specifically for this situation, which is why ScanGrid is implemented as an Action rather than a topic-based loop or a service call.

2.4 A Real Toolchain Issue Found and Fixed

During development, a genuine ROS 2 Jazzy rosidl toolchain bug was diagnosed: an Action's Result field that referenced another custom message defined in the *same* package (RiskClassification[] inside ScanGrid's Result) caused the interface generator to crash. This was confirmed by isolating the field and rebuilding. The permanent fix - and arguably better design - was to remove the large array from the Action's Result entirely and instead stream per-cell results live over the existing risk_classification topic, which both avoids the bug and gives a better live-updating demo experience than a single large result delivered only at the end.

3. The Deep Learning Part of the Project

3.1 What the Model Does

The DL component is a small feed-forward neural network (a Multi-Layer Perceptron, or MLP) that takes a single measured resistivity value (ρ , in ohm-metres) and predicts one of four corrosion-risk categories. It runs inside the risk_classifier_node, which subscribes to the resistivity topic and publishes a RiskClassification message (label + confidence) for every reading it receives.

3.2 Model Architecture

Layer	Shape	Activation
Input	1 (resistivity only)	-
Hidden (Linear)	1 -> 8	ReLU
Output (Linear)	8 -> 4	Softmax (via CrossEntropyLoss during training)

Framework: PyTorch (CPU build). Optimizer: Adam (lr=0.01). Loss: CrossEntropyLoss.

3.3 The Four Output Classes

Labels are based on the **Langford & Broomfield** resistivity classification, a standard reference used alongside the four-probe (Wenner) method that ResistiMap physically implements:

Resistivity Range	Label
< 50 ohm-m	very_high

50 - 100 ohm-m	high
100 - 200 ohm-m	low_moderate
> 200 ohm-m	low

3.4 Why Resistivity Is the Only Input

The demo slabs are built with known, deliberately varied moisture and salt content, so those values could technically be used as extra model inputs. They are deliberately **not** used that way. The entire purpose of ResistiMap is to infer corrosion risk from resistivity alone, because a real concrete structure's moisture/salt content is exactly the unknown the system is meant to estimate - it is never available in the field. Moisture and salt percentages are used only to generate the correct training *label* for each slab reading, never as a model input, so the trained model remains usable outside the lab.

3.5 Training Data: Synthetic Placeholder vs. Real Data

Two distinct datasets are involved, and it is important to keep them separate:

- **Synthetic (throwaway) data:** 2,000 randomly generated resistivity values, each labeled using the Langford & Broomfield rule above. This was used only to verify that the full pipeline - training, saving, loading, and live prediction inside a ROS 2 node - works correctly before any hardware existed. The resulting model (model_synthetic.pt) is a placeholder and is not the final result.
- **Real data (the actual deliverable):** resistivity readings physically measured by ResistiMap's four-probe sensor on demo concrete slabs whose moisture/salt content was deliberately controlled and known in advance. Each real reading is labeled using the same Langford & Broomfield rule, based on the known condition of the slab it was taken from. This is the dataset the final, reported model is trained and evaluated on.

3.6 Where the DL Model Sits in the Full System

resistivity_sensor_node publishes a Resistivity message for each cell -> risk_classifier_node loads the trained MLP (currently the synthetic placeholder), runs a forward pass, and publishes a RiskClassification message with the predicted label and confidence -> heatmap_builder_node consumes that message to update the live color-coded grid shown during the demo.